E-COMMERCE PLATFORM — SYSTEM DESIGN & INTERVIEW REFERENCE V1

Scalable, available, consistent where it must be — with the reasoning, the trade-offs and the failures
requirements · capacity · CAP per capability · checkout path · inventory · payments · API · production reality

1 · HOW TO ANSWER THIS — REQUIREMENTS, SCOPE AND THE FIRST FIVE MINUTES

start here — what to establish before drawing anything, because the design follows from these numbers

Clarify first — do not start drawing

Ask	Why it changes the design
Scale?	DAU, catalogue size, orders/day, peak multiple. Decides sharding, caching and whether this is one region or several
Read:write ratio?	E-commerce is typically **~100:1 or higher** browse-to-buy. That single fact justifies most of the caching and the read replicas
Flash sales?	If yes, the hot-key and oversell problems dominate the design. If no, they are edge cases
Marketplace or single seller?	Multi-tenant changes isolation, search, payouts and settlement entirely
Geography?	Multi-region introduces data residency, replication lag and a much harder consistency story
Guest checkout?	Decides whether session state is mandatory before identity exists

The most common interview failure is not a wrong component — it is designing for unstated scale. Sharding a database that holds 40 GB, or hand-waving a flash sale that is the actual hard part. Name the numbers, then design to them.

Functional scope — state what is in and out

In	Browse and search catalogue · product detail · cart · checkout · payment · order lifecycle · inventory · reviews · recommendations · seller catalogue management
Out	Warehouse management, logistics routing, pricing/promotion engine internals, fraud scoring models, returns and refunds workflow — *named explicitly, so the design has an edge*

Non-functional targets — with percentiles

Target	Value
Product page	**p99 < 200 ms** server-side
Search	p99 < 300 ms
Add to cart	p99 < 150 ms
Checkout submit	p99 < 1 s (external payment call dominates)
Availability — browse	99.99%
Availability — checkout	99.95% is often the honest number, because it depends on a payment provider you do not control
Durability — orders	No acknowledged order may be lost. This is non-negotiable

Always say "p99", never "average". An average hides a bimodal distribution completely, and it is the tail that customers abandon carts over.

Availability arithmetic worth knowing

SLA	Downtime / year
99%	3.65 days
99.9%	8.8 hours
99.99%	**52.6 minutes**
99.999%	5.3 minutes — very expensive, and rarely justified for retail

Serial dependencies multiply, and this is the number people get wrong. A checkout that must call five services each at 99.9% has a composite availability of roughly 99.5% — about 44 hours a year. Quoting 99.99% for a system with a long synchronous dependency chain is arithmetic, not ambition.

Which is why the design removes dependencies from the critical path. Every service you can make asynchronous, cache, or degrade gracefully around raises the composite number. That is the real reason for the event-driven parts of this architecture — not decoupling for its own sake.

The order to work through it

1. Clarify and scope → 2. Estimate → 3. Define the API
2. Data model and access patterns → 5. High-level architecture
3. Deep-dive the two hard parts: **inventory** and **payment**
4. Bottlenecks, failure modes, degradation → 8. Trade-offs you would revisit

2 · CAPACITY ESTIMATION

the numbers that justify the design

```
// stated assumptions — say these out loud
DAU                        10,000,000
sessions/user/day                 1.5
page views/session                 20
                  → 300 M views/day

average RPS   300M / 86,400   ≈  3,500 rps
peak (evening, ×5)            ≈ 17,500 rps
flash sale (×50 for minutes) ≈ 175,000 rps

// write path is far smaller
conversion ~2%   → 300,000 orders/day
                 → ~3.5 orders/sec average
                 → ~35/sec at peak

// read:write ≈ 100:1 — the single most
// important ratio in the whole design
```

Storage

Catalogue	10 M products × ~5 KB = **~50 GB**. Fits in memory on one machine — this is *not* big data, and does not need sharding
Orders	300 K/day × 2 KB × 365 = **~220 GB/year**, growing linearly. Partition by time
Images	10 M × 5 × 200 KB = **~10 TB** — object storage plus a CDN, never a database
Events	300 M/day × 1 KB = **~300 GB/day** into Kafka. This dominates the infrastructure bill

What these numbers tell you. Reads are 100× writes, so cache aggressively and replicate reads. The catalogue is small enough to keep hot in memory. The order database does not need sharding at this scale for years. **The events, not the orders, are the volume problem.**

Do the arithmetic before choosing technology. Half the components on a typical e-commerce diagram are unjustified at 3,500 rps and 50 GB — and the same diagram is badly under-designed for the flash sale, which is where the real engineering is.

3 · CAP AND PACELC — PER CAPABILITY, NOT PER SYSTEM

the framing that is usually stated wrongly, and what each part of the platform actually chooses

The correction first: CAP is not "pick two of three". You do not choose **P** — network partitions happen whether you want them or not, so any distributed system must tolerate them. The real statement is narrower: **during a partition**, you must choose consistency or availability. When there is no partition, you get both, and Brewer himself has said the two-of-three formulation was always misleading.

PACELC is the version you actually use. If **P**artition → choose **A** or **C**; **E**lse → choose **L**atency or **C**onsistency. Partitions are rare; the latency-versus- consistency trade-off is made on *every single request*, which makes it the more useful lens day to day.

And it is chosen per capability, not per system. The same platform is CP for payment and inventory, and AP for catalogue and reviews — often within one request. Answering "our system is AP" is the sign of someone who has read about CAP rather than applied it.

Capability	Choice	Why, and what it costs
Payment & ledger	CP	A double charge or lost payment is unrecoverable trust damage. Refuse the write rather than risk divergence. Single-region primary, synchronous commit
Inventory decrement	CP	Overselling has a real cost — cancellations, compensation, reputation. Serialise on the item, or hold a reservation
Order placement	CP	An acknowledged order must exist. Durability before response
User account & auth	CP	Credential and permission changes must not be stale — a revoked session must be revoked
Product catalogue	AP	A price or description seconds out of date is acceptable; the storefront being down is not. Cache and replicate freely
Search index	AP	Near-real-time is fine. Serving slightly stale results beats returning an error
Reviews & ratings	AP	A review appearing a minute later harms nobody
Recommendations	AP	Stale recommendations degrade quality slightly; unavailable ones remove a whole surface
Cart	Mixed	AP while browsing — never lose a cart, tolerate a stale line. **Becomes CP at checkout**, when price and stock are re-validated authoritatively

The cart row is the interesting one, and worth volunteering. The same data deliberately changes consistency class at a business boundary — cheap and available while shopping, strict at the moment money is involved. That is what "per capability" means in practice.

A second correction worth making

MongoDB is commonly labelled AP. **By default it is CP** — a replica set has one primary, writes with majority write concern are confirmed by a majority, and the minority side of a partition has no primary and therefore cannot accept writes. It is *tunable* toward availability with weaker read and write concerns, which is exactly the point: the classification follows the configuration, not the product.

4 · REFERENCE ARCHITECTURE

synchronous read path on top · write and event path below · every store chosen for one access pattern

EDGE & SYNCHRONOUS READ PATH — 100× the traffic of the write path, and the reason most of this exists

Clients — web · mobile · seller portal · guest and signed-in

→ **CDN + WAF** — static, images, HTML · DDoS, bot, TLS · *absorbs most read traffic*

→ **API gateway** — authN · rate limit · quota · routing · versioning · *one place for cross-cutting*

SERVICES — bounded by data ownership, not by noun. Each owns its store; none reads another's database.

Catalogue read-heavy · AP · cached	**Search** own index · AP	**Cart** session · AP → CP	**Inventory** CP · the hard one	**Order** CP · state machine	**Payment** CP · idempotent · PSP
User / identity CP · sessions	**Pricing / promo** read-heavy · cached	**Review** AP · moderated	**Recommendation** AP · precomputed	**Notification** async only	**Seller / fulfilment** back-office

STORES — polyglot, but each choice must be justified by an access pattern

PostgreSQL orders · payments · users	**Document store** catalogue · variable attrs	**Redis** cart · session · counters	**OpenSearch** search · facets
Object storage images · invoices	**ClickHouse** analytics · clickstream	**Vector index** similar items · semantic	**Read replicas** reporting · exports

THE CACHING LADDER — where the 100:1 ratio is actually absorbed

1 · **Browser** — static assets, immutable filenames
2 · **CDN** — images, JS/CSS, and cacheable product HTML fragments
3 · **Gateway** — short-TTL responses for anonymous traffic
4 · **Application** — Redis: product, price, session, cart
5 · **Database** — buffer pool and query cache

Each layer should absorb an order of magnitude. If the database sees product-page traffic at all, a layer above it is misconfigured.

CHECKOUT — THE SYNCHRONOUS CRITICAL PATH, AND WHAT IS DELIBERATELY PUSHED OFF IT

1 · Validate cart — re-price from source · re-check stock · *never trust client totals*
→ **2 · Reserve stock** — atomic decrement of available, TTL 15 min · *hold, do not deduct*
→ **3 · Create order** — status = PENDING + outbox event · *one local transaction*
→ **4 · Authorise payment** — idempotency key · external PSP call · *the slowest step*
→ **5 · Confirm** — order = CONFIRMED · reservation → committed · *the pivot step*
→ **6 · Respond** — order id + status · *target p99 < 1 s*

PUSHED OFF THE CRITICAL PATH — all asynchronous, via events
Confirmation email · invoice generation · warehouse notification · loyalty points · analytics · recommendation update · search reindex · fraud re-scoring · seller payout
Every one of these removed from step 6 raises composite availability and cuts p99.

WHAT MAKES THIS SAFE
Steps 2–4 are a **saga**: if payment fails, release the reservation and cancel the order. Reservation TTL means a crashed checkout self-heals — stock returns without intervention.
Order is created *before* payment, so a successful charge always has somewhere to land.

THE ORDER STATE MACHINE — every transition is an event, and the terminal states are explicit

```
PENDING → AWAITING_PAYMENT → CONFIRMED → PACKED → SHIPPED → DELIVERED → COMPLETED
   |            |                |
   |            |                └→ RETURN_REQUESTED → REFUNDED
   |            └→ CANCELLED      └→ PAYMENT_FAILED └→ CANCELLED (with refund)
```

THE EVENT BACKBONE — how everything above stays in step without synchronous coupling

Outbox in each service writes the state change and its event in one transaction → **CDC or a relay** publishes to Kafka → consumers apply it idempotently via an **inbox**.
Topics partitioned by aggregate id, so per-order and per-item ordering is preserved. A schema registry versions the contracts. A dead-letter queue captures what cannot be processed.
This is what lets the search index, recommendation models, analytics warehouse and seller dashboards all derive from the same facts without any of them being in the checkout path.

CROSS-CUTTING — the parts that decide whether this can be operated, not merely drawn

IDENTITY & SECURITY
· OIDC, short-lived tokens, refresh rotation; guest sessions upgraded on signup
· mTLS or workload identity between services — network position is not authentication
· PCI scope minimised: card data goes to the PSP, never through your services
Tokenised card references only. Never store a PAN or CVV.

OBSERVABILITY
· Trace per request, correlated across the async hops via correlation id
· RED per service; business metrics: conversion, cart abandonment, oversell count
· SLO burn-rate alerting; deployment markers on every dashboard
Alert on checkout success rate — it is the only metric that means revenue.

RESILIENCE
· Timeout, retry with jitter, circuit breaker and bulkhead on every remote call
· Fallbacks defined per dependency — see the degradation matrix
· Load shedding at the gateway, prioritising checkout over browse under stress
Shed browse traffic before checkout. Revenue over pageviews.

DELIVERY
· Build once, promote the same artifact; canary with automated rollback
· Expand → migrate → contract for every schema change
· Feature flags so release is decoupled from deploy; kill switch per surface
Deployment freeze windows around peak trading events.

CONTINUITY
· Multi-AZ by default; multi-region for read traffic and disaster recovery
· RTO and RPO stated per data class — orders and payments are the strictest
· Backups tested by restoring, and the restore timed
Replication is not a backup. It copies a bad DELETE faithfully.

COST
· Events are the largest line — set Kafka retention deliberately, not by default
· CDN offload ratio is the single highest-leverage cost metric
· Right-size non-production; it is routinely half the bill
Track cost per order as an engineering metric, not a finance one.

READING THE DIAGRAM — the three design decisions everything else follows from

1 · Reads and writes are separated by two orders of magnitude, so they get separate machinery. The read path is cached at five layers and served from replicas and derived indexes; the write path is small, strict and transactional. Optimising them together produces a system that is expensive for reads and unsafe for writes.

2 · Consistency is chosen per capability, and it changes within a single user journey. Browsing is AP and cached; the moment the customer clicks pay, the same cart data is re-validated against the authoritative source under CP semantics. The boundary between them is the checkout submit, and it is deliberate.

3 · Everything that does not have to be synchronous is not. Six steps are on the critical path; nine more happen afterwards from events. This is what makes the composite availability arithmetic work, and it is why the outbox and the event backbone are load-bearing rather than decorative.

A useful way to interrogate any box on this diagram: what access pattern justifies it, what is its p99, what happens to the customer when it is down, and what would you remove first if the budget halved?
If a component has no answer to all four, it is on the diagram because it is fashionable rather than because it is needed — and at 3,500 rps most e-commerce platforms carry several of those.

5 · WHY EACH COMPONENT — THE ANSWER, AND THE FOLLOW-UP

what an interviewer is testing is whether you know the cost, not the benefit

Component	Why · and what it costs you
CDN	**Why:** images and static assets are most of the bytes and none of the logic. Serving from the edge cuts latency and origin load together. **Cost:** cache invalidation becomes a distributed problem; use immutable filenames and version the URL rather than purging
WAF / DDoS	**Why:** a public storefront is permanently under scraping and credential-stuffing traffic. **Cost:** false positives block real customers; tune in monitor mode before enforcing
API gateway	**Why:** one place for authentication, rate limiting, quotas and versioning, instead of each service reimplementing them. **Cost:** a single point of failure and a potential bottleneck — it must scale horizontally and stay stateless
Microservices	**Why:** catalogue and checkout have completely different scaling and availability profiles; separate ownership and deployment. **Cost:** a distributed system, with every problem on this sheet. **A modular monolith is the right answer more often than it is chosen**
Kafka	**Why:** absorbs spikes, decouples producers from consumers, and lets many derived systems read the same facts with replay. **Cost:** operational weight; ordering only per partition; consumer lag becomes a thing you monitor and page on
Redis	**Why:** sub-millisecond cart, session, counters and rate limits. **Cost:** it is **AP with async replication and can lose acknowledged writes on failover** — never the source of truth for money or stock

Component	Why · and what it costs you
PostgreSQL	**Why:** orders, payments and inventory need real transactions, constraints and isolation. Nothing else on the sheet is a substitute. **Cost:** single-writer scaling; plan partitioning and connection pooling early
Document store	**Why:** catalogue items have wildly varying attributes by category — a fixed schema fights the domain. **Cost:** no joins; you denormalise, and you own keeping duplicates in step. **PostgreSQL JSONB removes this component for many teams**
OpenSearch	**Why:** relevance ranking, facets, typo tolerance and autocomplete are a different problem from querying rows. **Cost:** a second copy of the catalogue to keep in step, and it must never be the source of truth
Object storage	**Why:** images and invoices are large, immutable and cheap to serve. **Cost:** egress charges; lifecycle policies matter
Service mesh	**Why:** mTLS, retries and circuit breaking uniformly, without touching application code. **Cost:** substantial. **Only adopt when you genuinely have many services** — below roughly a dozen, libraries are simpler
Kubernetes	**Why:** bin-packing, autoscaling, rolling deploys and self-healing. **Cost:** a platform to operate. Managed compute is often sufficient and dramatically cheaper in engineering time

The answer that distinguishes a staff-level response. For every component, state what you would need to see before adding it, and what you would remove first. "We use Kafka because it decouples services" is a brochure. "We use Kafka because eight consumers need the same order events with replay, and below that a database table and a poller would do" is engineering.

6 · INVENTORY & OVERSELL

the hardest correctness problem

State the problem precisely. Ten thousand customers click buy on the last 100 units within the same second. Read-then-write without serialisation means many of them read "100 available" and all of them succeed. Overselling is a business incident: cancellations, compensation, and in some jurisdictions a regulatory one.

Three levels of stock

On hand	Physically in the warehouse
Reserved	Held for in-flight checkouts, with an expiry
Available	`on_hand − reserved` — the only number a customer may act on

Making the decrement safe

```
-- atomic and correct: the DB serialises it
UPDATE inventory
   SET available = available - :qty
 WHERE sku = :sku
   AND available >= :qty;
-- 0 rows affected = insufficient stock, cleanly

-- and the reservation, with an expiry
INSERT INTO reservation(order_id, sku, qty, expires_at)
VALUES (…, now() + interval '15 minutes');
```

Why not a distributed lock? A lock with a TTL is not mutual exclusion — a stalled holder can have its lease expire while it still believes it holds it, giving two writers. A conditional update inside the database is simpler, cheaper and actually correct.

Flash sale — the hot-key problem

- **One row becomes the bottleneck.** Every buyer contends on the same SKU, and lock waits collapse throughput
- **Shard the counter** — split 1,000 units into 10 buckets of 100 and pick one at random. Contention drops tenfold; the cost is a small chance of a false sold-out while another bucket has stock
- **Admission control** — a queue or token gate in front, so only as many requests reach inventory as there is stock to sell
- **Fail fast and honestly** when sold out; do not queue a customer behind an outcome that is already decided

The expiry is what makes it self-healing. A crashed checkout releases its stock automatically. Without a TTL, every abandoned cart leaks inventory permanently and someone reconciles it by hand.

7 · PAYMENTS

where mistakes are unrecoverable

The governing fact. A timeout from a payment provider tells you **nothing** about whether the customer was charged. The request may have succeeded and the response been lost. Every design decision below follows from that one sentence.

Non-negotiables

- **Idempotency key per payment attempt**, generated when the intent is formed and sent to the PSP. A retry with the same key returns the original result rather than charging again
- **Order created before payment**, so a successful charge always has a record to attach to
- **Money as integer minor units** with an explicit currency. Never floating point
- **An append-only ledger** — payment state is derived from entries, not overwritten in place
- **Card data never touches your services.** Tokenise at the PSP; this is what keeps PCI scope small

The states, and the awkward one

```
INITIATED → AUTHORISED → CAPTURED → SETTLED
    |            |           |
    |            |           └→ REFUNDED
    |            └→ VOIDED
    └→ FAILED

UNKNOWN  ← timeout. Not a failure. Must be
           resolved by query or reconciliation,
           never by assuming and retrying blindly.
```

Webhooks and reconciliation

- **Verify the signature** on every webhook; treat the payload as untrusted otherwise
- **Webhooks are at-least-once and can arrive out of order** — deduplicate by event id and ignore transitions older than current state
- **Never trust a webhook alone.** Poll the PSP for anything left in `UNKNOWN`
- **Daily reconciliation** against the provider's settlement file, with a report of every mismatch. This is how double charges are actually caught

Authorise then capture, rather than charging immediately. Authorisation holds the funds while you confirm stock and fraud checks; capture on dispatch. It makes cancellation cheap and is what most retail flows do.

8 · CACHING & SEARCH

absorbing the 100:1 ratio

Invalidation — the actual hard part

Strategy	When
Short TTL	Default. Simple, self-correcting, and bounded staleness you can state
Event-driven	Price and stock changes publish an event; consumers evict. Fast, and the invalidation itself can fail
Versioned key	Include a version in the key so a new version is a cache miss rather than a delete race
Write-through	For data that must never be stale — rare on the read path here

Three failure modes that cause real outages. Stampede — a popular key expires and thousands of requests hit the database at once; fix with jittered TTLs and a single-flight lock. **Hot key** — one product saturates one Redis node; fix by replicating the key across nodes or caching in-process. **Cold start** — after a restart the cache is empty and the database receives traffic it was never sized for; warm before taking traffic.

Search indexing pipeline

```
catalogue write → outbox → Kafka → indexer → OpenSearch
                                          └→ vector index

// never dual-write to DB and index directly:
// that is the dual-write problem, and it silently
// leaves the index wrong with no error
```

The index is a projection, never the source of truth. It must be fully rebuildable from the catalogue, and rebuilding should be a routine operation you have actually practised — because you will need it after a mapping change.

Hybrid retrieval is now standard: keyword search for exact terms, model numbers and brand names; vector search for intent and synonyms. Fuse the two ranked lists. Keyword alone fails on "something warm for winter"; vector alone fails on a part number.

9 · DEGRADATION MATRIX

what still works when X is down

Decide this at design time, not during the incident. Each row is a deliberate product decision about what the customer sees.

Down	Behaviour
Search	Fall back to category browse and the cached bestseller list. **Never a blank page**
Recommendations	Hide the widget entirely. It is decoration, and a spinner is worse than an absence
Reviews	Show the cached aggregate rating; hide the review list
Cache	Serve from the database with a circuit breaker and load shedding. Expect a hard capacity drop and shed browse first
Inventory	**Stop accepting checkout.** Selling stock you cannot verify is worse than a clear error
Payment provider	Offer an alternative provider if integrated; otherwise save the cart, tell the truth, and invite the customer back
Order service	Checkout is down. Keep browse fully available — it is a different failure domain by design
Notification	Nothing customer-visible. Events queue and drain later
Analytics	Nothing customer-visible. Never let analytics block a request

The principle running through the table. Browse degrades gracefully; checkout fails loudly and safely. Never take money you cannot honour, and never show a customer an empty page when a cached approximation exists.

Test these paths deliberately. Fallbacks are the least-exercised code in the system and run precisely when everything else is already failing. A fallback that has never been executed is a hypothesis.

10 · API SPECIFICATION

the contract, including the parts usually left off — status codes, pagination, concurrency and errors

Method	Endpoint	Auth	Notes & status codes
POST	`/v1/auth/register`	none	**201** · 409 if email exists · rate limited hard
POST	`/v1/auth/login`	none	**200** + access & refresh token · 401 · 429 after n attempts
GET	`/v1/products`	optional	**200** · **cursor** pagination, not offset · `Cache-Control` · `ETag`
GET	`/v1/products/{id}`	optional	**200** · 404 · `ETag` for conditional GET
GET	`/v1/search`	optional	**200** · `q, filters, facets, sort, cursor` · degrades to catalogue browse on failure
GET	`/v1/cart`	session	**200** · works for guest via session id
POST	`/v1/cart/items`	session	**200** · 409 if unavailable · returns the recalculated cart
PATCH	`/v1/cart/items/{id}`	session	**200** · `If-Match` for optimistic concurrency
POST	`/v1/orders`	bearer	**201** · **`Idempotency-Key` required** · 409 stock · 422 price changed
GET	`/v1/orders/{id}`	bearer	**200** · 403 if not the owner
POST	`/v1/orders/{id}/cancel`	bearer	**202** — asynchronous · 409 if already shipped
POST	`/v1/payments`	bearer	**202** · `Idempotency-Key` required · returns `PENDING`
GET	`/v1/payments/{id}`	bearer	**200** · poll for terminal state
POST	`/v1/webhooks/psp`	signature	**200** always, after verifying signature · dedupe by event id
POST	`/v1/reviews`	bearer	**202** — queued for moderation, not published immediately

Conventions applied to every endpoint

Versioning	In the path. A breaking change is a new version; existing consumers keep working
Pagination	**Cursor, not offset.** Offset pagination drifts when data changes underneath and degrades badly at depth
Idempotency	Required on every unsafe retryable POST. Stored with the result; a repeat returns the original response, not a conflict
Concurrency	`ETag` + `If-Match` on mutable resources, returning **412** on mismatch
Rate limits	`X-RateLimit-*` headers and **429** with `Retry-After` — tell the client how to behave
Tracing	`X-Request-ID` accepted and echoed; propagated to every downstream hop

One error envelope, everywhere

```
HTTP/1.1 409 Conflict
{
  "code": "INSUFFICIENT_STOCK",
  "message": "Only 2 units available",
  "requestId": "req_8f3c…",
  "details": [{ "sku": "A1", "available": 2 }]
}
```

A stable machine-readable code is what clients branch on — never the human message, which is free to change and to be localised.

Note the status codes on the write path. `POST /orders` returns **201** synchronously because the customer must know the order exists; cancel and payment return **202** because they complete asynchronously. Choosing between them honestly is more informative than any response body.

Place an order — the request that matters

```
POST /v1/orders
Authorization: Bearer eyJ…
Idempotency-Key: 0138F3…   ← client-generated,
X-Request-ID: req_8f3c…       one per intent
{
  "cartId": "c_9912",
  "addressId": "a_44",
  "paymentMethodId": "pm_tok_77",
  "expectedTotal": { "amount": 249900,
                     "currency": "INR" }
}

HTTP/1.1 201 Created
{
  "orderId": "o_1183",
  "status": "PENDING",
  "total": { "amount": 249900, "currency": "INR" },
  "payment": { "id": "pay_552", "status": "PENDING" },
  "reservationExpiresAt": "2026-09-01T14:15:00Z"
}
```

`expectedTotal` is the important field. The server re-prices authoritatively and compares. A mismatch returns **422** with the new total rather than silently charging a different amount — which is what turns a stale-price bug into a handled case.

Money is an object, not a number. Integer minor units plus an explicit currency. A bare `2499.00` in JSON invites floating-point handling somewhere down the chain.

The reservation expiry is returned to the client so the interface can show a countdown. Making the hold visible is what stops customers believing checkout has silently failed.

11 · PRODUCTION PROBLEMS — WHAT ACTUALLY HAPPENS

the questions an interviewer asks after the happy path, and the answers that show you have operated one of these

Problem	What you see	Answer
Overselling	Orders confirmed for stock that does not exist; cancellations and compensation	Conditional atomic decrement, reservations with TTL, sharded counters for hot SKUs
Double charge	Customer charged twice on a retry after a timeout	Idempotency key end to end, PSP-side idempotency, daily reconciliation
Lost order	Payment captured, no order record	Create the order before charging; outbox so nothing depends on a dual write
Cache stampede	Latency spike and a database CPU cliff when a popular key expires	Jittered TTL, single-flight lock, early recompute of hot keys
Hot partition	One Kafka partition or one Redis node saturated during a sale	Better partition key, key splitting, local caching of the hot item
Consumer lag	Search and dashboards silently minutes behind; nobody notices for hours	Alert on lag, not just on errors; scale consumers with partitions
Connection exhaustion	Database refuses connections while CPU is idle	Pooling with a bounded upper limit; a proxy in front; do not scale pods without scaling pool maths

Problem	What you see	Answer
Retry storm	A slow dependency triggers retries that make it slower still	Exponential backoff **with jitter**, circuit breaker, retry budget
Thundering herd on deploy	New pods start cold and time out under full traffic	Warm caches and pools before readiness passes; ramp traffic
Stale price shown	Customer sees one price, is charged another; complaints and chargebacks	Re-price authoritatively at checkout; honour the shown price within a small tolerance and alert on the delta
Search index drift	Products missing or wrong; no error anywhere	Rebuild from the source; reconcile counts nightly; alert on index age
Migration lock	A schema change locks a large table and takes checkout down	Expand → migrate → contract; build indexes concurrently; test at production data size
Flash sale collapse	Everything falls over in the first ninety seconds	Pre-scale, queue admission, shard the counter, shed browse traffic, cache aggressively, and load test at the real number
Stuck saga	Orders parked in an intermediate state; discovered when a customer calls	Alert on anything exceeding its expected window; DLQ with an owner and a runbook

What breaks first — the bottleneck ladder

Be ready for this follow-up. Naming the sequence, and how each is detected, is more convincing than any diagram.

#	Limit	Detected by
1	**Database connection pool.** Saturates long before CPU does, and scaling pods makes it worse	Connection wait time; refused connections while CPU is idle
2	**Hot SKU row** during a sale — lock contention on one row collapses throughput	Lock wait time; p99 on one endpoint only
3	**Cache capacity and cold start** — eviction rate climbs, hit ratio falls, load lands on the database	Hit ratio; eviction rate; origin request rate
4	**Kafka consumer lag** on derived systems — silent, and it degrades search and dashboards first	Lag per consumer group
5	**Payment provider rate limit** — external, and you cannot scale past it	429s from the PSP; checkout success rate
6	**Single-writer database throughput** — the last one, and the one people design for first	Write latency; replication lag

Note the order. The limit people plan for — write throughput — is last. The one that actually takes systems down is first, and it is a configuration number rather than a hardware one.

The trade-offs I would revisit. Whether the document store earns its place over JSONB; whether a mesh is justified at this service count; whether Kafka retention is set from cost rather than from replay need. Volunteering what you would reconsider is a stronger close than defending every choice.

12 · TECHNOLOGY CHOICES — ADVANTAGES, DISADVANTAGES AND THE ALTERNATIVE

every row includes what you would use instead, because "why not X?" is the second question

Component	Choice	Why this one	Advantages	Disadvantages	Alternative, and when
Transactional store	PostgreSQL	Orders, payments and inventory need real transactions, constraints and isolation levels	ACID, mature, JSONB, full-text, geospatial, pgvector — often removes three other components	Single writer; vertical scaling limits; needs pooling and partitioning discipline	**Distributed SQL** when write throughput genuinely exceeds one node — measured, not projected
Catalogue store	PostgreSQL JSONB, or a document store	Category-varying attributes fight a fixed schema	Flexible schema; fast development; horizontal scaling in the document case	A second store to operate; no joins; you own denormalisation	**Start with JSONB** in the database you already run. Add a document store only when it demonstrably falls short
Cache & session	Redis	Sub-millisecond cart, session, counters and rate limiting	Rich types — sorted sets cover leaderboards and rate limits; very fast	**AP: can lose acknowledged writes on failover.** Memory cost; hot keys; eviction tuning	**Memcached** for a pure cache. For durable cart, back it with the database
Search	OpenSearch / Elasticsearch	Relevance, facets, typo tolerance and autocomplete	Mature, well understood, scales horizontally, strong aggregations	Heavy to operate; a second copy of the catalogue; eventually consistent	**PostgreSQL full-text** below roughly a million products — it removes an entire system
Event backbone	Kafka	Retention and replay let many derived systems read the same facts independently	High throughput, durable, ordered per partition, replayable	Operationally heavy; ordering only per partition; lag monitoring is mandatory	**SQS / RabbitMQ** if you need a work queue rather than a replayable log — simpler, and usually enough
Analytics	ClickHouse	Aggregating clickstream and order data over billions of rows	Extremely fast scans and compression; SQL	Poor at point updates; batch-oriented; another system	**The warehouse you already have**, or read replicas, until the scan volume justifies it
Object storage	S3 or equivalent	Images and invoices are large, immutable and cheap	Effectively unlimited, durable, integrates with CDN	Egress cost; eventual listing semantics; lifecycle policies needed	None sensible. Do not put images in a database
Recommendations	Precomputed item-to-item + vector	Read path must be a lookup, not a computation	Fast, simple, strong baseline, easy to reason about	Refreshed on a schedule, so new items lag	**Graph database** only if traversal depth is genuinely the requirement — it is a heavy choice for "also viewed"
Service comms	REST external, gRPC internal, events for the rest	Right protocol per boundary	REST is universal and cacheable; gRPC is fast and typed; events decouple	Three styles to maintain and document	**REST everywhere** is a perfectly defensible simplification at this scale
Orchestration	Kubernetes	Bin-packing, autoscaling, rolling deploys, self-healing	Portable, standard, rich ecosystem	A platform to operate; genuine complexity; easy to over-adopt	**Managed container services** — materially less to run, and sufficient for most teams
Service mesh	Istio or Linkerd	Uniform mTLS, retries and circuit breaking without code changes	Consistent policy, strong telemetry, traffic splitting	Substantial operational and cognitive cost; another failure domain	**Libraries and the gateway** below roughly a dozen services. Adopt a mesh when you actually have the scale

The honest summary an interviewer is listening for. At 3,500 rps and a 50 GB catalogue, a well-tuned PostgreSQL, Redis, a CDN and a modular monolith would serve this workload with far less operational surface. Everything above is justified by **organisational** scale — many teams deploying independently — as much as by traffic. Saying so is a strength, not a concession.

13 · READINESS CHECKLIST

before this platform takes real orders and real money

- ☐ **Scale, read:write ratio and peak multiple** written down and agreed
- ☐ **Latency targets stated at p99**, per endpoint
- ☐ **Composite availability computed** for the checkout dependency chain
- ☐ **CAP choice recorded per capability**, not per system
- ☐ **Cart re-validated at checkout** — price and stock from the source
- ☐ **Inventory decrement is a conditional atomic update**, not read-then-write
- ☐ **Reservations expire**, and expiry rate is monitored
- ☐ **Hot-SKU strategy exists** — sharded counter or admission control
- ☐ **Order created before payment** is attempted
- ☐ **Idempotency key on every unsafe POST**, carried end to end
- ☐ **Repeat requests return the original result**, not a conflict
- ☐ **Money stored as integer minor units** with currency
- ☐ **Payment `UNKNOWN` state modelled** and resolved by query, not assumption
- ☐ **Webhook signatures verified**; events deduplicated and order-tolerant
- ☐ **Daily settlement reconciliation** with a mismatch report
- ☐ **No card data in your systems** — tokenised at the provider
- ☐ **Outbox in every service that publishes**; no dual writes anywhere
- ☐ **Every consumer idempotent**, with dedupe in the same transaction
- ☐ **Topics partitioned by aggregate id** where ordering matters
- ☐ **Consumer lag alerted on**, not only errors
- ☐ **DLQ owned, monitored and drainable**
- ☐ **Search index rebuildable**, and the rebuild practised
- ☐ **Cache TTLs jittered**; single-flight on hot keys; cold-start plan
- ☐ **CDN offload ratio measured** and tracked
- ☐ **Degradation matrix agreed** with product, and each path tested
- ☐ **Load shedding prioritises checkout** over browse
- ☐ **Timeout, retry with jitter, circuit breaker** on every remote call
- ☐ **Connection pool sized** against database limits and replica count
- ☐ **Schema changes are expand → migrate → contract**, timed at production size
- ☐ **Load tested at flash-sale volume**, not at average
- ☐ **RTO and RPO defined**; restore tested and timed
- ☐ **PII and payment data handling reviewed** against the applicable regime
- ☐ **Checkout success rate alerted on** as the primary business SLI
- ☐ **Runbooks** for oversell, double charge, stuck saga, provider outage, flash sale
- ☐ **Cost per order tracked**, with the largest contributors known

The one that summarises the sheet: **walk one order from click to delivery and name, at every hop, what happens if that hop fails — and whether the customer keeps their money, their order, or neither.** If any step answers "it would probably retry", that is the part of the design that is not finished.